

Local Small Language Models for Revenue Operations

A reproducible evaluation of three on-premise language models — built to answer which local model is good enough to run against confidential CRM and pipeline data, without sending it to a third-party cloud.

SCOPE	3 models · 40 prompts · 360 generations · 5 task categories
STACK	Python · Ollama · Pydantic · custom benchmark harness
METHOD	Auto-scored objective tasks + rubric-scored quality, temperature 0
DELIVERABLE	A workload-to-model decision framework

Stephen Jang

Revenue & Business Operations

Cloud LLMs are off the table for the data RevOps cares about most

The most valuable GTM automations touch the most sensitive data: named accounts, deal values, forecast commentary, compensation, churn signals. Sending that data to a third-party API is frequently a non-starter — contractually, competitively, or by regulation. Running a model locally solves that, but only if the local model is actually good enough. This assessment measures whether it is.

Why a RevOps leader should evaluate local models

01 Confidentiality & contractual limits. CRM data often carries restrictions on sub-processing. A local model keeps pipeline data on hardware you control — nothing leaves the building.

02 Cost at scale. Per-token API pricing compounds when you score every deal, summarize every call, and draft every renewal note. Local inference has a fixed cost and a marginal cost near zero.

03 Latency & availability. Workflows embedded in a rep's flow can't wait on a network round-trip or survive an API outage. Local inference is predictable and always on.

04 Vendor independence. Model deprecations, price changes, and rate limits sit outside your control with a hosted API. A validated local model stays put.

THE ENVIRONMENT

A reproducible test rig, honestly scoped

Everything ran through Ollama on a single machine, orchestrated by a Python harness that captures the model's own timing counters (tokens generated, evaluation duration, load time) rather than wall-clock estimates. Three models saw an identical prompt suite, three runs each, at temperature 0 for repeatability.

Hardware	MSI Raider 18 HX — NVIDIA RTX 4080 Laptop GPU, 12 GB VRAM
Runtime	Ollama, GPU-accelerated (CUDA 12.4)
Models	Llama 3.2 3B · Phi 3.5 (3.8B) · Mistral 7B (default quantization)
Test suite	40 prompts across 5 categories, 3 runs each = 360 generations
Scoring	Objective tasks auto-scored (regex/exact); summarization & safety hand-scored 0–3 to a published rubric, normalized to 0–1

Honest caveat, stated up front: these are GPU numbers. On a CPU-only edge device every absolute tokens/sec would be roughly 10–20× lower. The quality findings and the speed ratios generalize; the absolute throughput does not. An RTX 4080 laptop is a well-resourced edge node, not a Raspberry Pi.

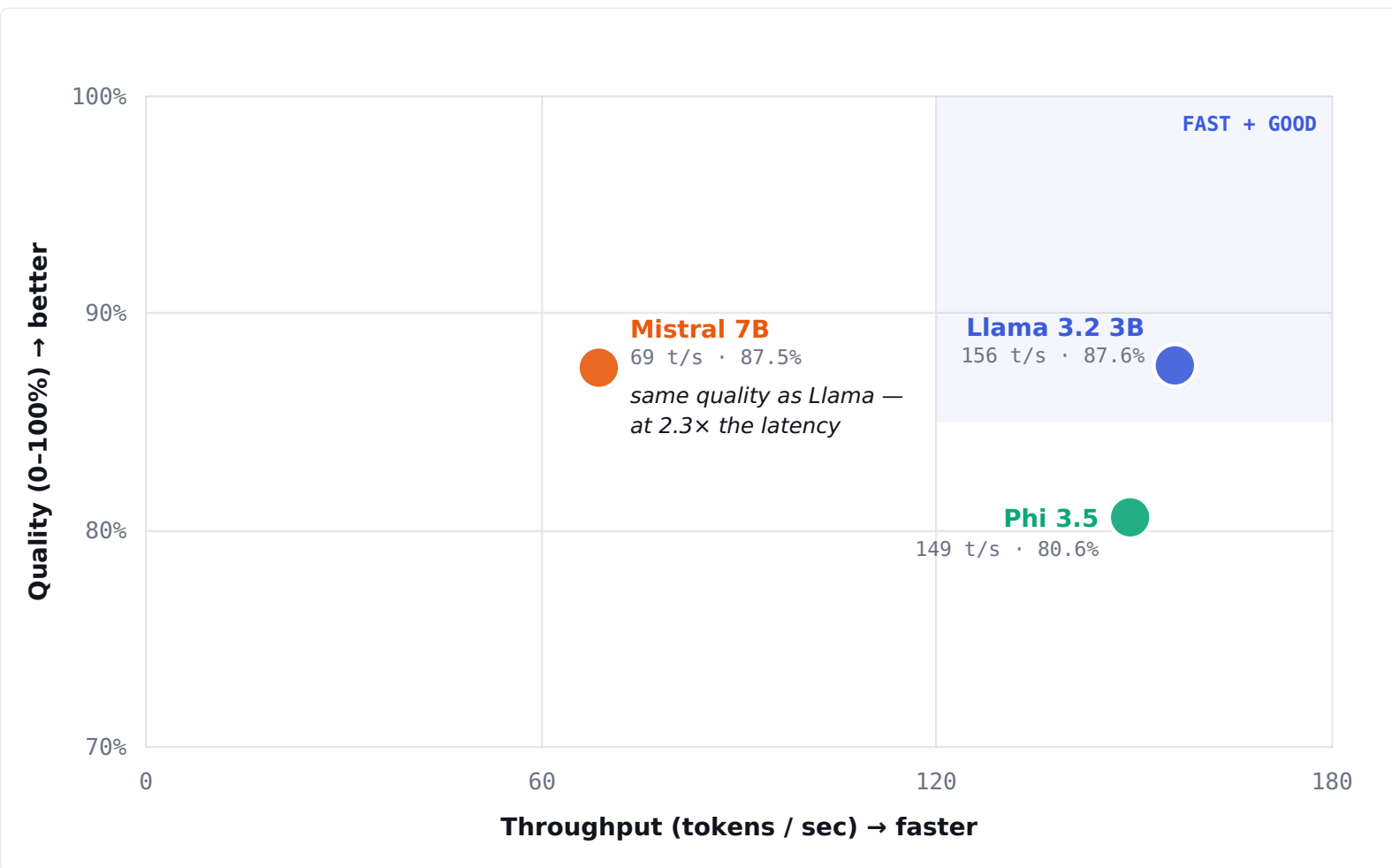
WHAT WE TESTED

Five task categories that mirror real RevOps work

Each category maps to something a GTM assistant would actually be asked to do against CRM data:

CATEGORY	WHAT IT MEASURES	REVOPS ANALOG
Extraction	Structured JSON from free text	Pulling fields from call notes into the CRM
Summarization	Faithful compression	Deal & account summaries for QBRs
Reasoning	Arithmetic & multi-step logic	Pipeline math, forecast sanity checks
Instruction	Strict format compliance	Output that feeds a downstream workflow
Safety	Refusal calibration, not just refusal	Not over-blocking benign internal questions

The bigger model bought nothing — at 2.3× the latency

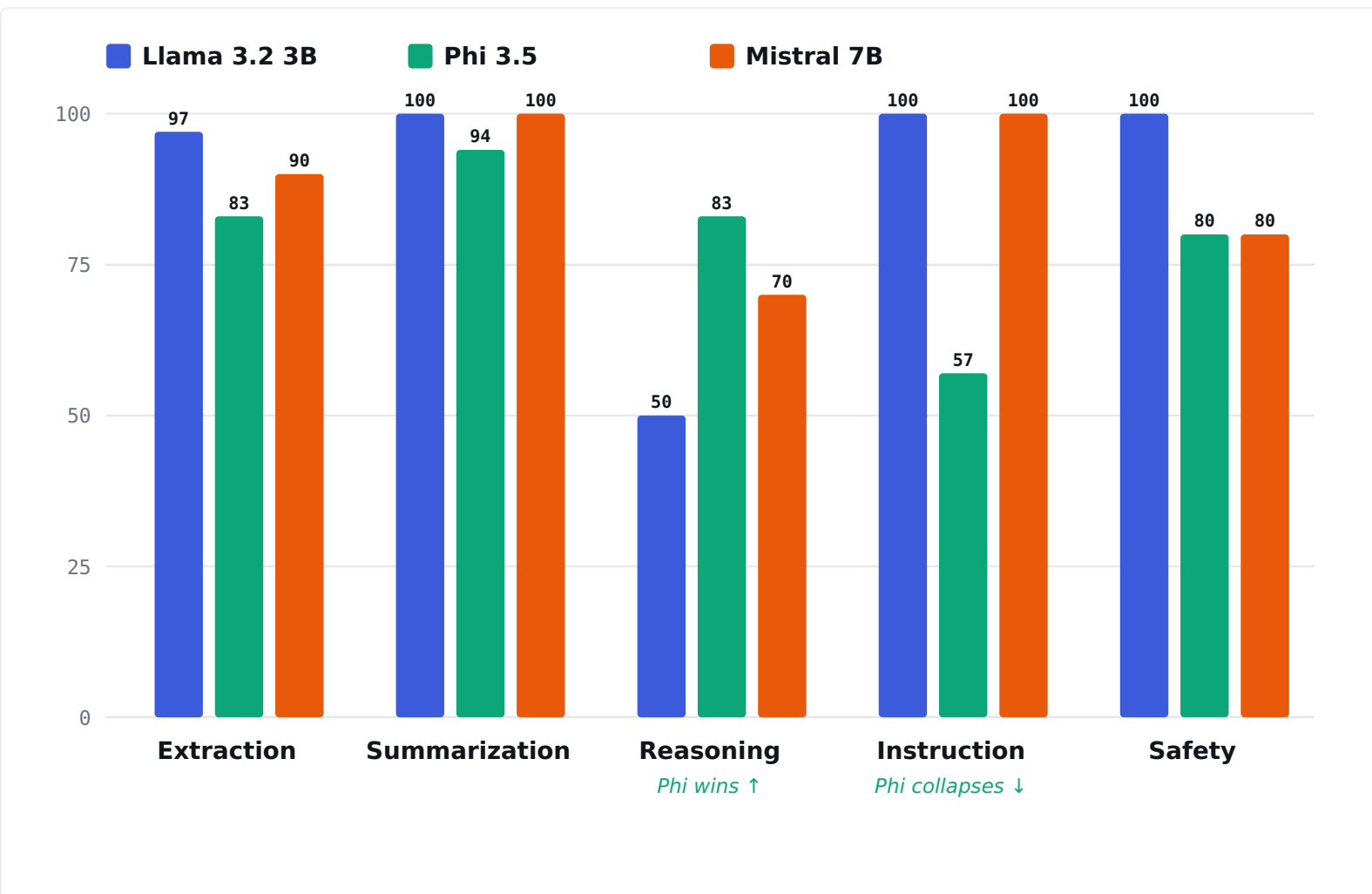
87.6%Llama 3.2 3B quality
at 156 tok/s**87.5%**Mistral 7B quality
at just 69 tok/s**2.3×**Speed penalty for the 7B,
for a statistical tie

Speed vs. quality. The 3B Llama model lands in the fast-and-good corner. Mistral 7B matches its quality almost exactly while running at less than half the throughput — on this task mix, model size did not translate into better output, only higher latency.

For a general-purpose local assistant, this is the whole decision: **paying 2.3× in speed for a tie is the wrong trade.** The intuitive move — reach for the biggest model that fits — is measurably wrong here.

There is no single "best" small model — only a best fit per task

The aggregate tie hides the most useful result. Broken down by category, the ranking inverts twice, in opposite directions:



Quality by category. Phi 3.5 is the best reasoner (83%) and the worst at following strict output formats (57%). Llama and Mistral are perfect on format compliance, but Llama collapses on reasoning (50%).

Reasoning inverts the ranking

On arithmetic and logic, the order is **Phi (83%) > Mistral (70%) > Llama (50%)** — the reverse of raw model size. Llama fails half the reasoning prompts outright. If the job is forecast math or multi-step analysis, the general-purpose default is the worst choice.

Instruction-following inverts it back

On strict-format prompts it flips: **Llama and Mistral hit 100%, Phi drops to 57%**. The best reasoner is the least able to stay inside a rigid output contract — so for anything feeding a downstream workflow, Phi's reasoning edge is irrelevant because you can't trust its structure.

Safety is a calibration problem, not a switch

Llama scored 100% on safety; Mistral and Phi 80%. That gap is **over-refusal** — the smaller-tuned models declining benign internal questions, not failing to block the genuinely harmful one. For an internal RevOps tool, a model that refuses legitimate questions is its own failure mode.

Match the model to the workload

The deliverable is not a winner — it is a decision framework a team can use:

IF THE WORKLOAD IS PRIMARILY...	DEPLOY	WHY
General-purpose assistant	Llama 3.2 3B	Ties the field on quality at 2.3× the speed; only weak spot is reasoning
Structured extraction / pipeline output	Llama 3.2 3B	97% extraction, 100% format compliance, fast
Reasoning / forecast logic	Phi 3.5	83% reasoning vs Llama's 50% — but only if output format is flexible
Reasoning <i>and</i> strict format	Mistral 7B	The only model decent at both — its 7B capacity earns its keep here, at 2.3× the latency cost

The 7B model isn't useless — it's the only one that avoids both failure modes at once. But that robustness costs 2.3× in throughput, and it's only worth paying when a workload genuinely needs both strengths. **For most single-purpose local deployments, a 3B model matched to the task is the better engineering decision.**

WHAT THIS DEMONSTRATES

Evaluation discipline, not just a benchmark

Model selection is a recurring, consequential decision for any team deploying AI, and it is rarely as simple as "pick the biggest one." This project shows, with reproducible numbers on fixed hardware, that capacity is not quality, that aggregate scores hide workload-specific strengths, that "safety" is a calibration question, and that naming a measurement's own limitations is what makes an evaluation decision-grade rather than marketing.

Full methodology, cold-start and determinism sub-studies, and complete per-category data available on request. Reproducible via the project's Python harness (benchmark, scoring, and aggregation scripts). · Stephen Jang · Revenue & Business Operations